

MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Balan Gabriel FAF-241

Report

Laboratory Work No. 9

Operating Systems

Topic: Watching System Calls with strace

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

1 Purpose of the Work 3

2 Part 1 – See the Calls of a Real Command 3

2.1 Observations 4

3 Part 2 – A Program that Calls the Kernel Directly 4

3.1 Observations 5

4 Part 3 – Trace Only the write System Call 6

4.1 Observations 7

5 Conclusion 7

1. Purpose of the Work

The purpose of this laboratory work is to observe directly how normal Linux programs communicate with the kernel through system calls. I used `strace` on a normal `ls -l` command, then wrote a small C program that uses `open`, `write`, `read` and `close` directly. This made the system-call layer much more visible, because I could compare what the C program asked for with what the kernel actually received.

2. Part 1 – See the Calls of a Real Command

For the first part I traced a normal `ls -l` command, limiting the output to the system calls related to opening files, reading, writing and closing descriptors.

Command

```
strace -f -e trace=openat,read,write,close ls -l
```

The trace immediately showed that even a very simple command performs many system calls before its normal output appears. A large part of the beginning is made of `openat`, `read` and `close` calls while the program loads libraries and accesses the information it needs.

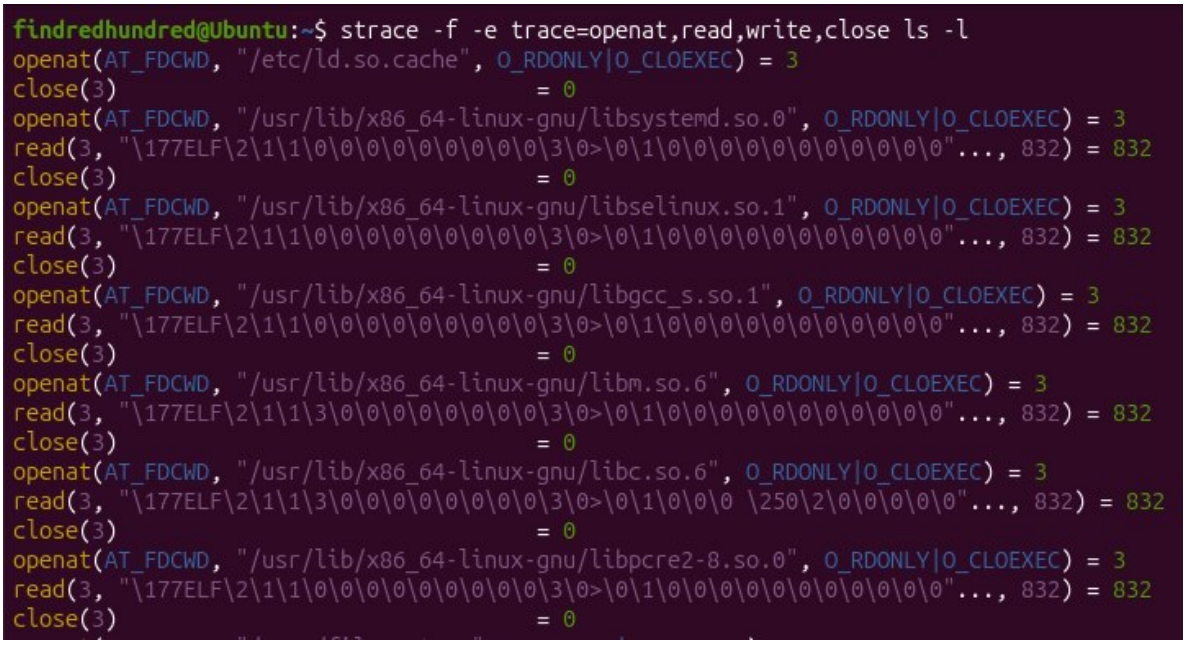


Figure 1.1 – Beginning of the `strace` output for `ls -l`.

Near the end of the trace, the normal directory listing is sent to the terminal through a `write` call. In my run, the important line was:

Relevant output

```
write(1, "total 56\ndrwxr-xr-x 2 findredhun"..., 977) = 977
+++ exited with 0 +++
```

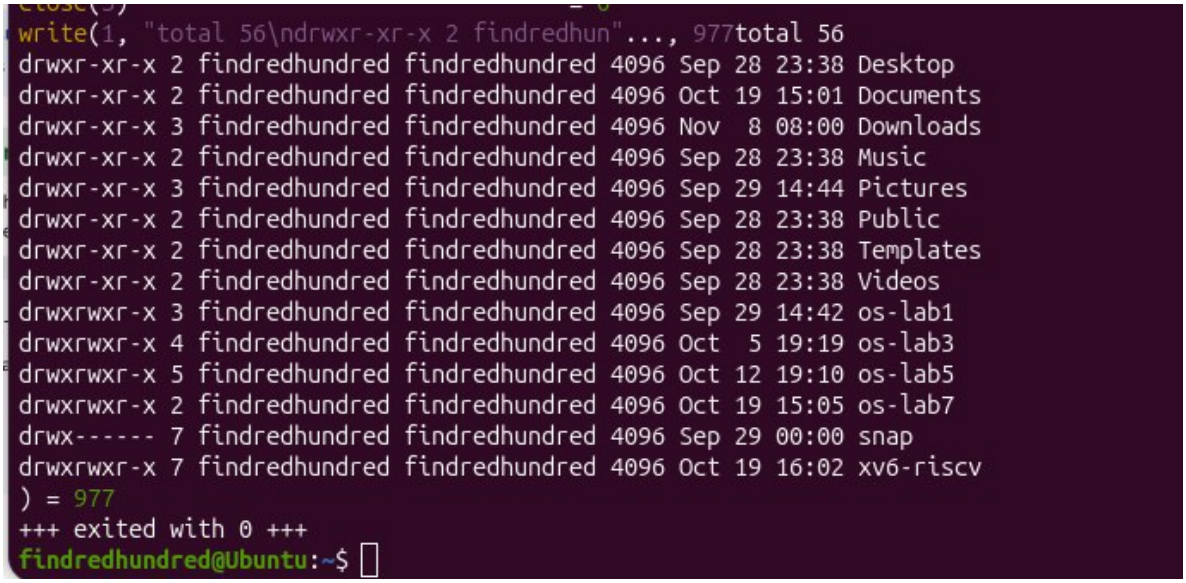


Figure 1.2 – `write` sends the directory listing to standard output.

2.1. Observations

On my system, `strace` showed roughly 70 system calls for one `ls` command. `ls` uses system calls such as `openat`, `read`, `write` and `close` to perform its task.

The system call that actually prints the listing to the screen is `write`. It writes to file descriptor `1`, which is standard output. This can be seen directly in the line `write(1, "total 56...", 977)`, where the first argument is the descriptor used for the output.

3. Part 2 – A Program that Calls the Kernel Directly

For the second part I used a short C program that opens a file, writes `hello, kernel` into it, closes it, opens it again, and reads the same data back. The file operations themselves use the system calls directly rather than using higher-level file functions.

syscalls.c

```
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <stdio.h>

int main(void) {
    const char *msg = "hello, kernel\n";

    int fd = open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }
    printf("open returned descriptor %d\n", fd);

    ssize_t n = write(fd, msg, strlen(msg));
```

```
    printf("write sent %zd bytes\n", n);
    close(fd);

    fd = open("note.txt", O_RDONLY);
    char buf[64];
    n = read(fd, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    printf("read got %zd bytes: %s", n, buf);
    close(fd);
    return 0;
}
```

I compiled and ran the program normally first.

Compile and run

```
gcc -Wall -Wextra -o syscalls syscalls.c
./syscalls
```

Program output

```
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
```

Then I ran the same program through `strace`:

Trace command

```
strace -e trace=openat,write,read,close ./syscalls
```

The interesting part of the trace was the sequence where `note.txt` was opened, written, closed, opened again, read, and closed again:

Relevant trace

```
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(3, "hello, kernel\n", 14) = 14
close(3) = 0
openat(AT_FDCWD, "note.txt", O_RDONLY) = 3
read(3, "hello, kernel\n", 63) = 14
close(3) = 0
```

3.1. Observations

The system call `open` returns the smallest available file descriptor. When a normal program starts, descriptors `0`, `1` and `2` are already occupied by standard input, standard output and standard error. Because of that, the first new file opened by this program receives descriptor

```
findredhundred@Ubuntu:~/os-lab9$ gcc -Wall -Wextra -o syscalls syscalls.c
findredhundred@Ubuntu:~/os-lab9$ ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
findredhundred@Ubuntu:~/os-lab9$ strace -e trace=openat,write,read,close ./syscalls
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0 \250\2\0\0\0\0\0"..., 832) =
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(1, "open returned descriptor 3\n", 27open returned descriptor 3
) = 27
write(3, "hello, kernel\n", 14)         = 14
write(1, "write sent 14 bytes\n", 20write sent 14 bytes
)   = 20
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_RDONLY)  = 3
read(3, "hello, kernel\n", 63)          = 14
write(1, "read got 14 bytes: hello, kernel"..., 33read got 14 bytes: hello, kernel
) = 33
close(3)                                = 0
+++ exited with 0 +++
```

Figure 2.1 – Running `syscalls` normally and tracing its file system calls with `strace`.

3, since it is the smallest number that is still available.

The trace also makes the relation between the C calls and the kernel much easier to see. The first `open` of `note.txt` becomes an `openat` system call returning descriptor 3, `write` sends exactly 14 bytes to that descriptor, and after closing it the next `openat` can reuse descriptor 3. The following `read` then gets the same 14 bytes back from the file.

4. Part 3 – Trace Only the write System Call

For the third part I limited `strace` to only the `write` system call. This makes the output much easier to follow because all other system calls are hidden and we can focus only on the moments when the program sends bytes either to the terminal or to a file.

Command

```
strace -e trace=write ./syscalls
```

The trace showed the following important calls:

Relevant output

```
write(1, "open returned descriptor 3\n", 27) = 27
write(3, "hello, kernel\n", 14) = 14
write(1, "write sent 14 bytes\n", 20) = 20
write(1, "read got 14 bytes: hello, kernel"..., 33) = 33
+++ exited with 0 +++
```

```
findredhundred@Ubuntu:~/os-lab9$ strace -e trace=write ./syscalls
write(1, "open returned descriptor 3\n", 27open returned descriptor 3
) = 27
write(3, "hello, kernel\n", 14)         = 14
write(1, "write sent 14 bytes\n", 20write sent 14 bytes
)   = 20
write(1, "read got 14 bytes: hello, kernel"..., 33read got 14 bytes: hello, kernel
) = 33
+++ exited with 0 +++
findredhundred@Ubuntu:~/os-lab9$
```

Figure 3.1 – Tracing only the `write` system call while running `syscalls`.

4.1. Observations

When running `strace -e trace=write ./syscalls`, we can clearly see the `write` system call being called before the output itself appears on the screen. For example, `write(1, ...)` is shown first by `strace`, and then the text appears in the terminal. This makes it very clear that printing to the screen is actually done through a system call to file descriptor `1`, which is standard output.

For the call `write(3, "hello, kernel \n", 14)`, there is no visible output on the screen. This is because file descriptor `3` is the opened `note.txt` file, so those 14 bytes are redirected into the file instead of being written to the terminal. The trace still shows the system call itself, but the actual text goes into the file.

5. Conclusion

This laboratory made system calls feel much less abstract because I could actually watch them happen. Even a normal command such as `ls -l` makes many calls to the kernel before producing its output, and the final listing reaches the terminal through `write` on descriptor 1. The small C program also showed why a newly opened file usually gets descriptor 3: descriptors 0, 1 and 2 are already reserved for the standard streams. Tracing only `write` also made it obvious that output sent to descriptor 1 appears on the terminal, while output sent to descriptor 3 goes into the opened file instead. Overall, `strace` made it much easier to connect normal C operations with the system calls that Linux performs underneath.